

MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer

Sachin Mehta and Mohammad Rastegari

ICLR 2022

2022.04.03

딥러닝논문읽기모임 이미지처리팀

홍은기, 김병현, 김선옥, 김현진, 박재완, 안종식, 이찬혁, 현청천

목차

1. Introduction
2. Proposal: MobileViT
3. Experimental Results
4. Ablations
5. Conclusion & Discussion

Introduction – Edge AI

Table 1. Left: Microcontrollers have 3 orders of magnitude *less* memory and storage compared to mobile phones, and 5-6 orders of magnitude less than cloud GPUs. The extremely limited memory makes deep learning deployment difficult. **Right:** The peak memory and storage usage of widely used deep learning models. ResNet-50 exceeds the resource limit on microcontrollers by 100×, MobileNet-V2 exceeds by 20×. Even the int8 quantized MobileNetV2 requires 5.3× larger memory and can't fit a microcontroller.

	Cloud AI (NVIDIA V100)	➔	Mobile AI (iPhone 11)	➔	Tiny AI (STM32F746)		ResNet-50	MobileNetV2	MobileNetV2 (int8)
Memory	16 GB	$\xrightarrow{4\times}$	4 GB	$\xrightarrow{3100\times}$	320 kB	← gap →	7.2 MB	6.8 MB	1.7 MB
Storage	TB~PB	$\xrightarrow{1000\times}$	>64 GB	$\xrightarrow{64000\times}$	1 MB	← gap →	102MB	13.6 MB	3.4 MB

MCUNet (NeurIPS 2020)

Introduction – Edge AI

Edge AI = Edge Computing + AI

1. Constraints

- 1) Limited Computing Power
- 2) Limited Memory
- 3) Limited Storage

2. Requirements (domain-specific)

- 1) Real-time inference (e.g., Self-driving car)
- 2) Robustness to harsh environment (e.g., Intelligent CCTV)

Introduction – Methodologies of Lightweight AI

1. Model Design

- a. EfficientNet
- b. ShuffleNet
- c. Mobilenet

2. Knowledge Distillation

3. Model Compression

- a. Quantization
- b. Pruning

4. Dedicated Hardware Design

- a. Google EdgeTPU (ASIC)
- b. FPGA

Key Question

Is it possible to **combine the strength of CNNs and ViTs** to build a **light-weight** and **low latency** network for mobile vision tasks?

As a result,

- MobileViT outperforms CNN and ViT-based networks
- MobileViT achieves top-1 accuracy of 78.4 % with about 6 million parameters on the ImageNet-1k

CNN pros & cons

- Pros
 - 1) Easy to train
 - 2) Fast & Light
 - 3) Spatial inductive bias
 - 4) Less sensitive to data augmentation
- Cons
 - 1) Local receptive field

ViT pros & cons

- Pros
 - 1) Global receptive field catches long-distance dependencies
 - 2) SOTA accuracy
- Cons
 - 1) Heavy parameters (under same parameter budget, DeiT performs worse than MobileNetV3)
 - 2) Expensive computation
 - 3) SLOW
 - 4) Sensitive to data augmentation and regularization

Related Works

- Light-weight CNNs
 - a. MobileNet (2017)
 - b. ShuffleNetV2 (2018)
 - c. ESPNetV2 (2019)
 - d. MixNet (2019b)
 - e. MNASNet (2019)
- Vision Transformers
 - a. ViT (2021)
 - b. ViT-C (2021)
 - c. CvT (2021)
 - d. PiT (2021)

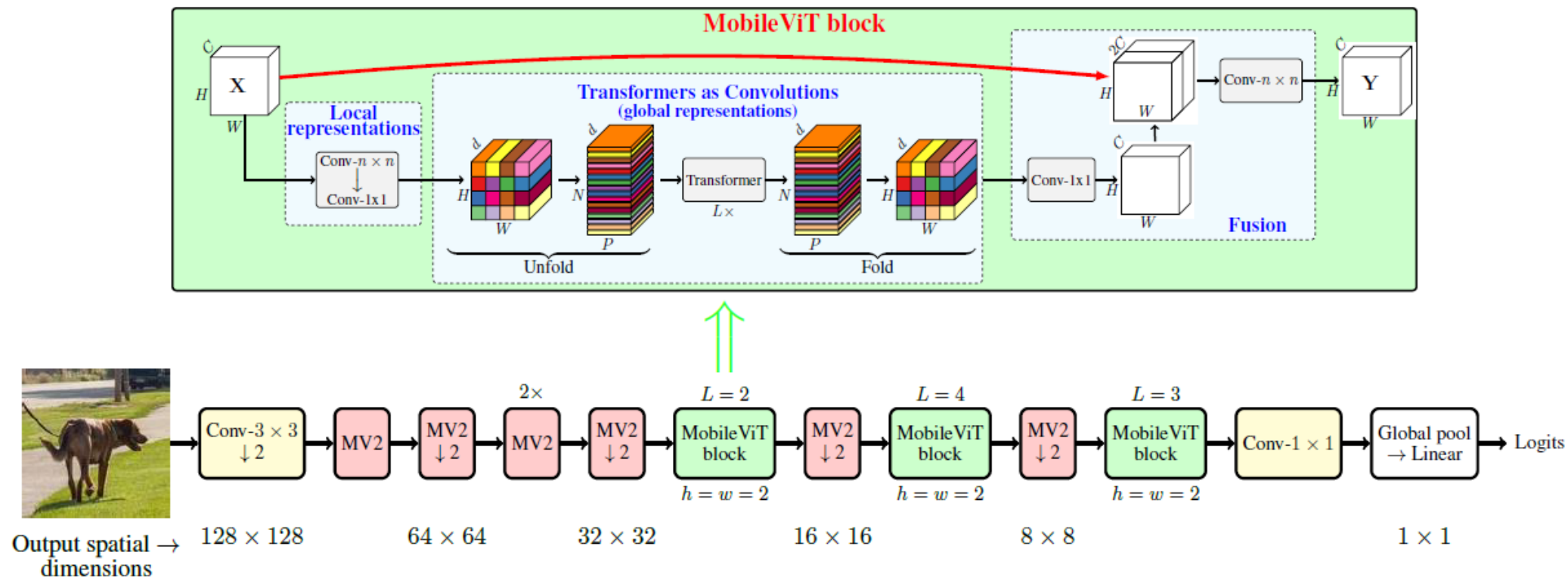
Proposal: MobileViT

Design Purpose

1. Better Performance - Under the same parameter budget, achieve better performance
2. Generalization Capability - small gap between training and evaluation metrics
 - Better performance on an unseen dataset.
3. Robust - to hyperparameters (e.g., data augmentation & L2 regularization)

Proposal: MobileViT

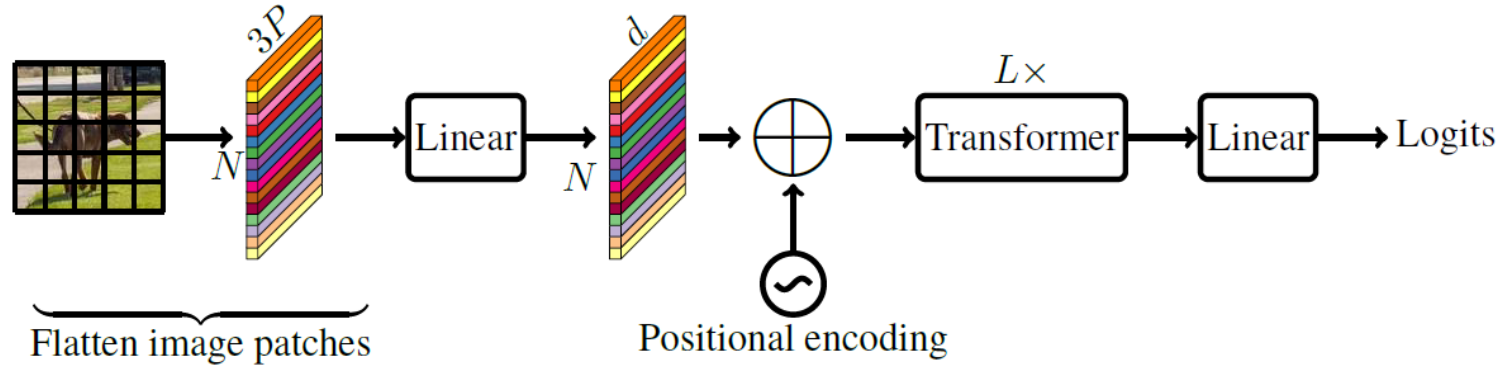
Architecture



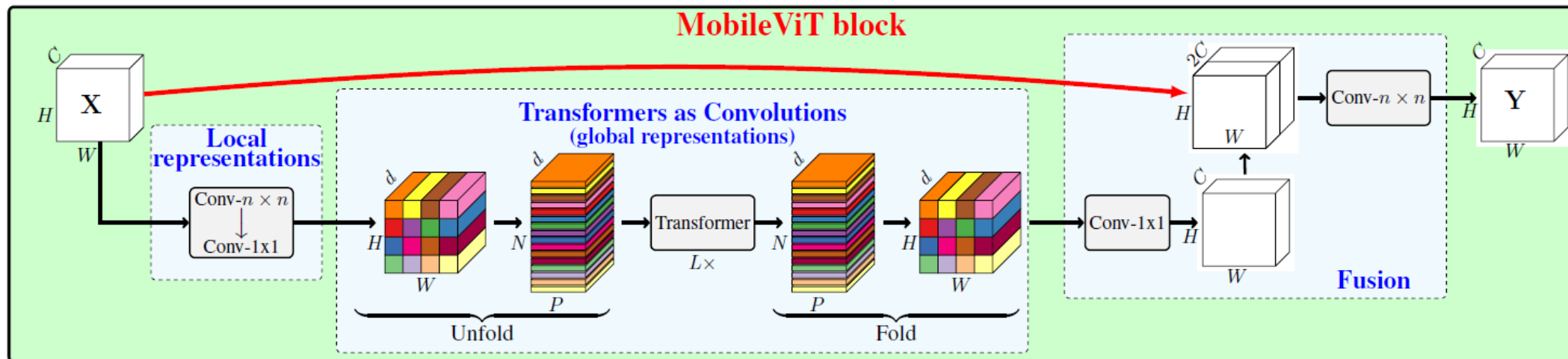
(b) **MobileViT**. Here, $\text{Conv-}n \times n$ in the MobileViT block represents a standard $n \times n$ convolution and **MV2** refers to MobileNetv2 block. Blocks that perform down-sampling are marked with $\downarrow 2$.

Proposal: MobileViT

Standard visual transformer (ViT)

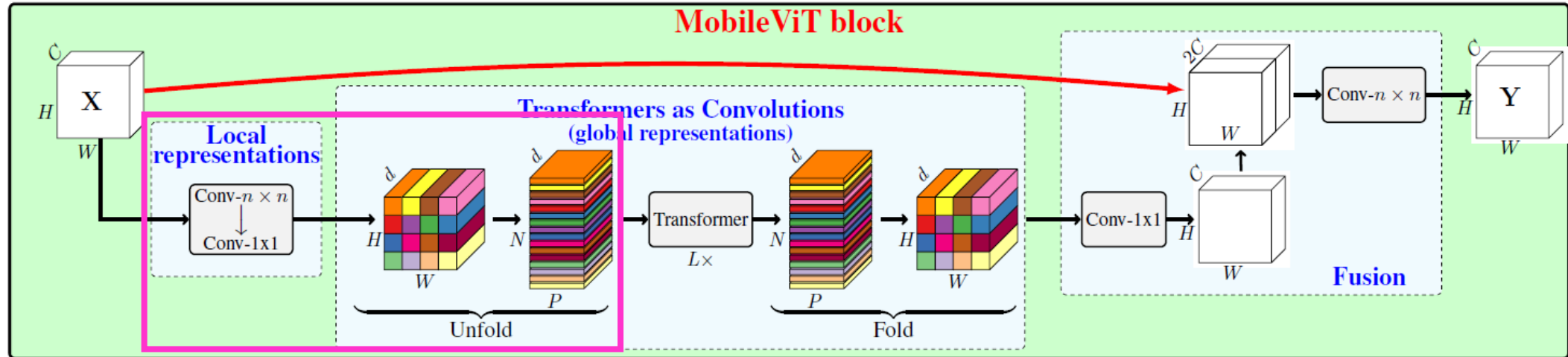


MobileViT Block



Proposal: MobileViT

MobileViT Block



(1) $X \in \mathbb{R}^{(H \times W \times C)} \rightarrow n \times n$ standard conv + 1×1 conv $\rightarrow X_L \in \mathbb{R}^{(H \times W \times d)}$

- d : high dimensional space ($d > C$)

(2) Unfold X_L into N non-overlapping flattened patches $X_U \in \mathbb{R}^{(P \times N \times d)}$

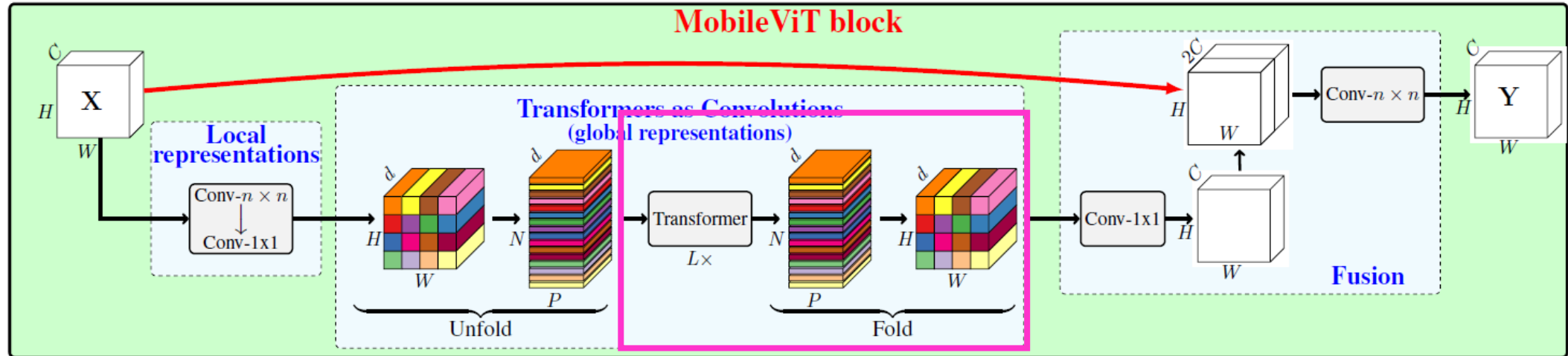
- patch size $P = wh$,

- number of patches $N = HW / P$

- height and width of patch: $h \leq n$, $w \leq n$

Proposal: MobileViT

MobileViT Block



(3) Apply transformer to encode inter-patch relationship for each $p \in \{1, \dots, P\}$:

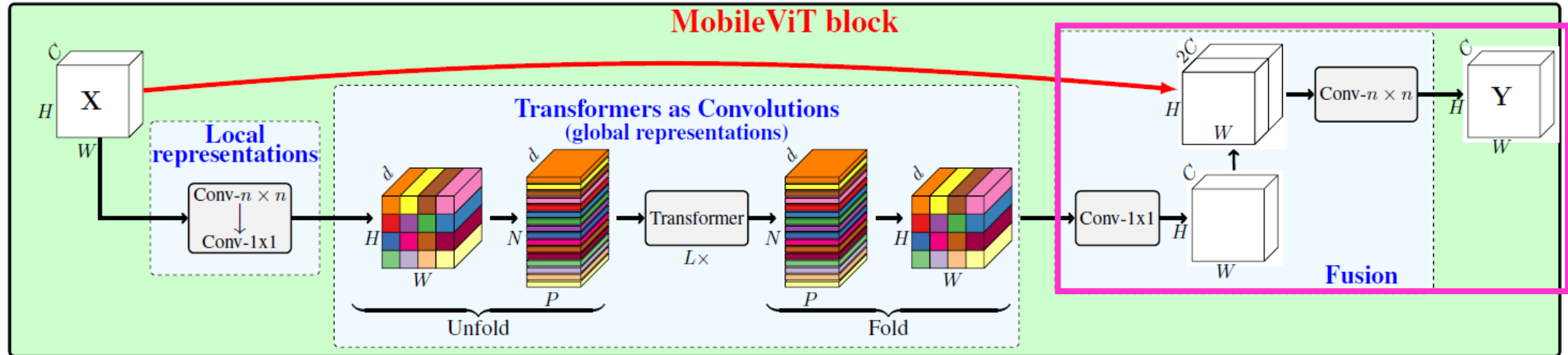
- $X_G(p) = \text{Transformer}(X_U(p))$, where $1 \leq p \leq P$
- $X_G \in \mathbb{R}^{(P \times N \times d)}$

(4) Fold $X_G \in \mathbb{R}^{(P \times N \times d)}$ to obtain $X_F \in \mathbb{R}^{(H \times W \times d)}$

- MobileViT neither loses the patch order nor the spatial order of pixels within each patch.

Proposal: MobileViT

MobileViT Block

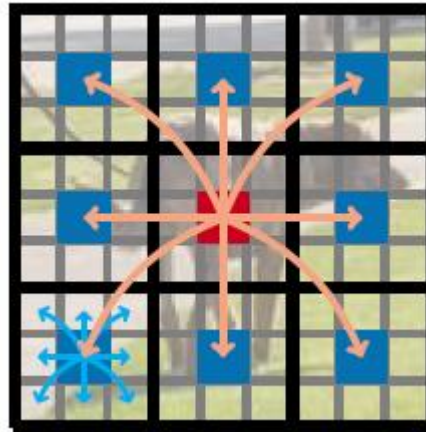


(5) Project X_F to low C -dimensional space using 1×1 conv and combine it with X via concat.

(6) Finally, apply another $n \times n$ conv to fuse the concatenated features.

Proposal: MobileViT

MobileViT Block & Global Computation



- **Blue pixels:** encodes information about the neighboring pixels using **convolution**.
- **Red pixel:** encodes information from all pixels using **self attention**.

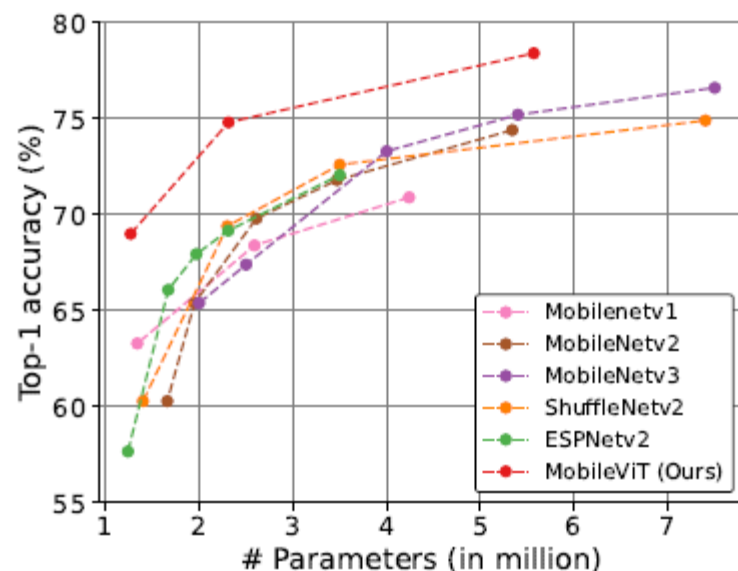
Proposal: MobileViT

Layer	Output size	Output stride	Repeat	Output channels		
				XXS	XS	S
Image	256×256	1				
Conv- $3 \times 3, \downarrow 2$	128×128	2	1	16	16	16
MV2			1	16	32	32
MV2, $\downarrow 2$	64×64	4	1	24	48	64
MV2			2	24	48	64
MV2, $\downarrow 2$	32×32	8	1	48	64	96
MobileViT block ($L = 2$)			1	48 ($d = 64$)	64 ($d = 96$)	96 ($d = 144$)
MV2, $\downarrow 2$	16×16	16	1	64	80	128
MobileViT block ($L = 4$)			1	64 ($d = 80$)	80 ($d = 120$)	128 ($d = 192$)
MV2, $\downarrow 2$	8×8	32	1	80	96	160
MobileViT block ($L = 3$)			1	80 ($d = 96$)	96 ($d = 144$)	160 ($d = 240$)
Conv- 1×1			1	320	384	640
Global pool	1×1	256	1			
Linear				1000	1000	1000
Network Parameters				1.3 M	2.3 M	5.6 M

Table 4: **MobileViT architecture.** Here, d represents dimensionality of the input to the transformer layer in MobileViT block (Figure 1b). By default, in MobileViT block, we set kernel size n as three and spatial dimensions of patch (height h and width w) in MobileViT block as two.

Q & A

Experiments – ImageNet -1k classification



(a) Comparison with light-weight CNNs

Model	# Params. ↓	Top-1 ↑
MobileNetv1	2.6 M	68.4
MobileNetv2	2.6 M	69.8
MobileNetv3	2.5 M	67.4
ShuffleNetv2	2.3 M	69.4
ESPNetv2	2.3 M	69.2
MobileViT-XS (Ours)	2.3 M	74.8

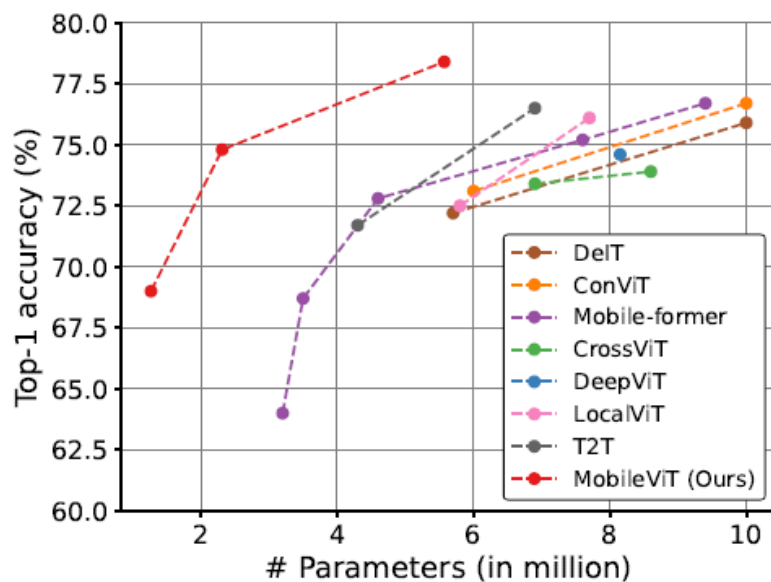
(b) Comparison with light-weight CNNs (similar parameters)

Model	# Params. ↓	Top-1 ↑
DenseNet-169	14 M	76.2
EfficientNet-B0	5.3 M	76.3
ResNet-101	44.5 M	77.4
ResNet-101-SE	49.3 M	77.6
MobileViT-S (Ours)	5.6 M	78.4

(c) Comparison with heavy-weight CNNs

Figure 6: **MobileViT vs. CNNs** on ImageNet-1k validation set. All models use basic augmentation.

Experiments – ImageNet -1k classification



(a)

Row #	Model	Augmentation	# Params. ↓	Top-1 ↑
R1	DeiT	Basic	5.7 M	68.7
R2	T2T	Advanced	4.3 M	71.7
R3	DeiT	Advanced	5.7 M	72.2
R4	PiT	Basic	10.6 M	72.4
R5	Mobile-former	Advanced	4.6 M	72.8
R6	PiT	Advanced	4.9 M	73.0
R7	CrossViT	Advanced	6.9 M	73.4
R8	MobileViT-XS (Ours)	Basic	2.3 M	74.8
R9	CeiT	Advanced	6.4 M	76.4
R10	DeiT	Advanced	10 M	75.9
R11	T2T	Advanced	6.9 M	76.5
R12	ViL	Advanced	6.7 M	76.7
R13	LocalViT	Advanced	7.7 M	76.1
R14	Mobile-former	Advanced	9.4 M	76.7
R15	PVT	Advanced	13.2 M	75.1
R16	ConViT	Advanced	10 M	76.7
R17	PiT	Advanced	10.6 M	78.1
R18	BoTNet	Basic	20.8 M	77.0
R19	BoTNet	Advanced	20.8 M	78.3
R20	MobileViT-S (Ours)	Basic	5.6 M	78.4

(b)

Figure 7: **MobileViT vs. ViTs** on ImageNet-1k validation set. Here, **basic** means ResNet-style augmentation while **advanced** means a combination of augmentation methods with basic (e.g., MixUp (Zhang et al., 2018), RandAugmentation (Cubuk et al., 2019), and CutMix (Zhong et al., 2020)).

Experiments – ImageNet -1k classification

Hyperparameter settings

1. Data: ImageNet-1k classification dataset
 - Train images: 1,280,000
 - Validation images: 50,000
2. Hardware: 8 NVIDIA GPUs
3. Batch size: 1,024
4. Optimizer: AdamW
5. Loss function: cross-entropy loss (label smoothing=0.1)
6. Multi-scale sampler: {(160, 160), (192, 192), (256, 256), (288, 288), (320, 320)}
7. Learning rate
 - 0.0002 to 0.002 for the first 3k iterations
 - annealed to 0.0002 using cosine annealing scheduler
8. L2 weight decay: 0.01
9. (Basic) data augmentation:
 - Random resized cropping
 - Horizontal flipping

Experiments – MobileViT as a general-purpose backbone

Feature backbone	# Params. ↓	mAP ↑
MobileNetv3	4.9 M	22.0
MobileNetv2	4.3 M	22.1
MobileNetv1	5.1 M	22.2
MixNet	4.5 M	22.3
MNASNet	4.9 M	23.0
MobileViT-XS (Ours)	2.7 M	24.8
MobileViT-S (Ours)	5.7 M	27.7

(a) Comparison w/ light-weight CNNs

Feature backbone	# Params. ↓	mAP ↑
VGG	35.6 M	25.1
ResNet50	22.9 M	25.2
MobileViT-S (Ours)	5.7 M	27.7

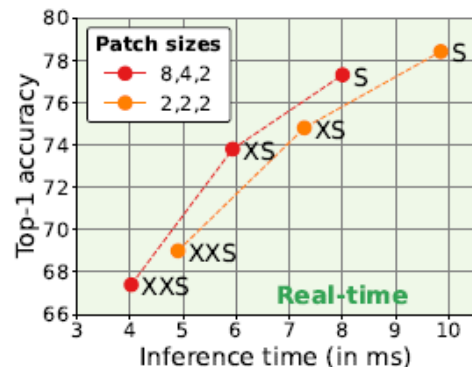
(b) Comparison w/ heavy-weight CNNs

Table 1: **Detection w/ SSDLite.**

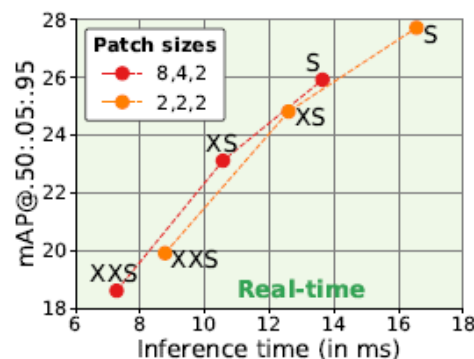
Feature backbone	# Params. ↓	mIOU ↑
MobileNetv1	11.2 M	75.3
MobileNetv2	4.5 M	75.7
MobileViT-XXS (Ours)	1.9 M	73.6
MobileViT-XS (Ours)	2.9 M	77.1
ResNet-101	58.2 M	80.5
MobileViT-S (Ours)	6.4 M	79.1

Table 2: **Segmentation w/ DeepLabv3.**

Experiments – Real-world on-device latency



(a) Classification @ 256×256



(b) Detection @ 320×320



(c) Segmentation @ 512×512

Figure 8: **Inference time of MobileViT models on different tasks.** Here, dots in green color region represents that these models runs in real-time (inference time < 33 ms).

Model	# Params. ↓	FLOPs ↓	Time ↓	Top-1 ↑
MobileNetv2 [†]	3.5 M	0.3 G	0.92 ms	73.3
DeiT	5.7 M	1.3 G	10.99 ms	72.2
PiT	4.9 M	0.7 G	10.56 ms	73.0
MobileViT (Ours)	2.3 M	0.7 G	7.28 ms	74.8

Table 3: **ViTs are slower than CNNs.**
[†]Results with multi-scale sampler (§B).

Experiments – Real-world on-device latency

Model	# Params ↓	FLOPs ↓	Top-1 ↑	Inference time ↓		
				iPhone12 - CPU	iPhone12 - Neural Engine	NVIDIA V100 GPU
MobileNetv2	3.5 M	0.3 G	73.3	7.50 ms	0.92 ms	0.31 ms
DeiT	5.7 M	1.3 G	72.2	28.15 ms	10.99 ms	0.43 ms
PiT	4.9 M	0.7 G	73.0	24.03 ms	10.56 ms	0.46 ms
MobileViT (Ours)	2.3 M	0.7 G	74.8	17.86 ms	7.28 ms	0.62 ms/0.47 ms [†]

MobileViT is 7.9x slower than MobileNetv2 with 2.3x larger FLOPs

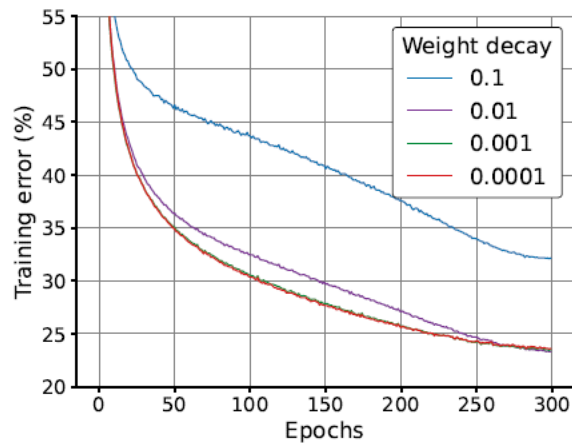
Why?

- Dedicated CUDA kernel exists for transformers on GPU, but not in mobile devices.
- CNN benefits from several device-level optimizations (including batch norm fusion with conv layers)

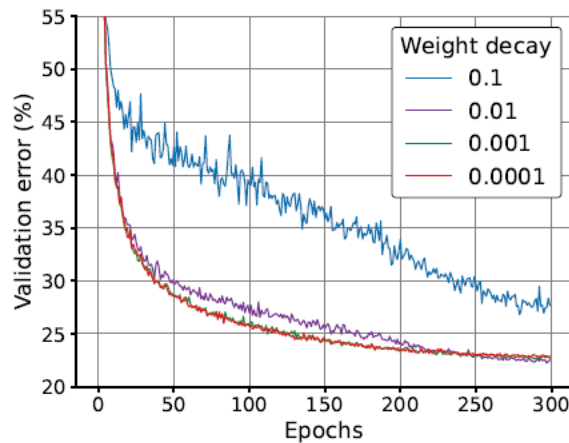
Ablations

1. Impact of weight decay

- A good model should be insensitive or less sensitive to L2 regularization (because tuning it for each task and dataset is time and resource consuming).
- Similar convergence except for a weight decay of 0.1



(a) Training error



(b) Validation error

Weight decay	Top-1 ↑
0.1	76.5
0.01	78.4
0.001	77.6
0.0001	77.4

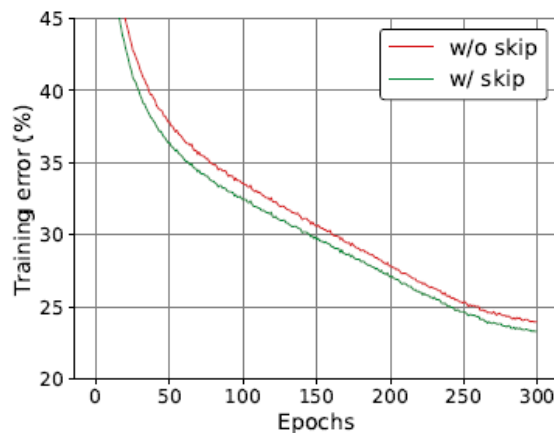
(c) Validation accuracy

Figure 11: **Impact of weight decay**. Here, results are shown for MobileViT-S model (5.7 M parameters) on the ImageNet-1k dataset. Results in (c) are with exponential moving average.

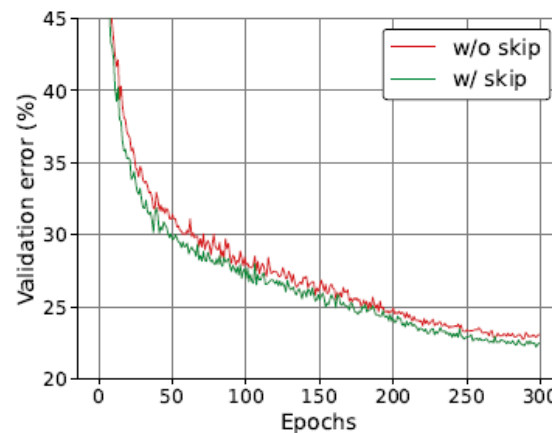
Ablations

2. Impact of skip-connection

- 0.5 % better performance with the skip-connection
- Outperforms other CNN and ViT based models even without the skip-connection



(a) Training error



(b) Validation error

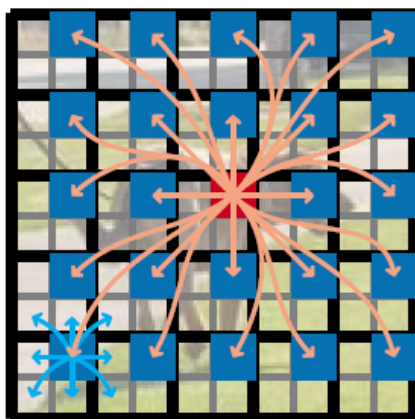
Skip-connection	Top-1 ↑
✗	77.9
✓	78.4

(c) Validation accuracy

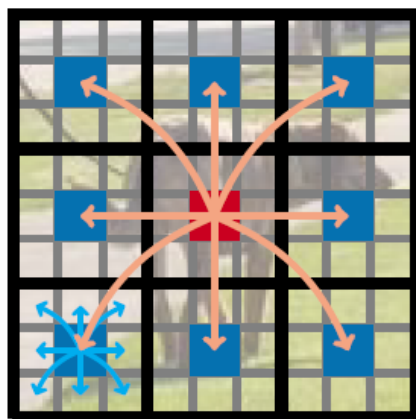
Figure 12: **Impact of skip connection.** Here, results are shown for MobileViT-S model (5.7 M parameters) on the ImageNet-1k dataset. Results in (c) are with exponential moving average.

Ablations

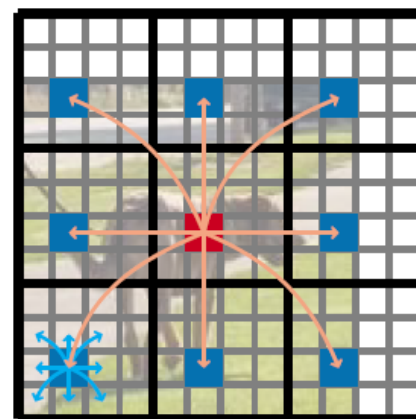
3. Impact of patch sizes



(a) $h = w = 2 < n = 3$



(b) $h = w = n = 3$



(c) $h = w = 4 > n = 3$

Patch sizes	# Params.	Time ↓	Top-1 ↑
2,2,2	5.7 M	9.85 ms	78.4
3,3,3 [†]	5.7 M	14.69 ms	78.5
4,4,4	5.7 M	8.23 ms	77.6
8,4,2	5.7 M	8.20 ms	77.3

Ablations

4. Impact of label-smoothing and EMA

LS	EMA	Top-1 ↑
✗	✗	78.0
✓	✗	78.3
✓	✓	78.4

Table 7: **Effect of label smoothing (LS) and exponential moving average (EMA) on the performance of MobileViT-S on the ImageNet-1k dataset.** First row results are with cross-entropy.

Conclusion & Discussion

1. Captures a global information with much less computation by combining CNN & ViT.
2. Is MobileViT mobile-friendly?
 - Kinda yes...
 - Higher accuracy than MobileNetV2 but much slower
 - Higher accuracy and Faster than other ViT-based models
 - May fit for a specific application under certain constraints.

Thank you